La pregunta clave que debes hacerte siempre

¿Cuáles son TODAS las decisiones que puedo tomar HOY, parado en este estado?

- No pienses en "el futuro", no pienses en "el resto". Solo en **hoy**. Si la respuesta es:
  - "Puedo **comprar** un **pase** de **1 día**" → una rama
  - "Puedo **comprar** un **pase** de **7 días**" → otra rama
  - "Puedo **comprar** un **pase** de **30 días**" → otra rama

Entonces la recursión es **exactamente**:

f(d) = min(costo_opción1 + f(**nuevo_estado1**),
costo_opción2 + f(**nuevo_estado2**),
costo_opción3 + f(nuevo_estado3))

¿Cada decisión a dónde me lleva? Me lleva a un nuevo estado, o lo que es lo mismo a un nuevo dia a resolver Día d+1, d+7, d+30

2. Cómo identificar el patrón (checklist)
Cuando leas un problema, pregúntate:

Pregunta	**En este problema**
¿Cuál es el estado?	El **día d** en el que **estoy**
¿Qué **decisiones** puedo **tomar** desde ese estado?	**Comprar** pase de 1, 7 o 30 días
¿Cada decisión cuánto **cuesta**?	costs[0], costs[1], costs[2]
¿Cada decisión a dónde me lleva?	Día d+1, d+7, d+30
¿Qué quiero?	**Minimizar** el costo total
¿Cómo combino?	**min**(...) de cada opción

Si respondes esas 6 preguntas, la recurrencia se escribe sola.

3. ¿Por qué min(...) y no +?

Esto es crucial:

- Si quieres **minimizar** → min
- Si quieres **maximizar** → max
- Si quieres **contar todas las formas** → suma +
- Si quieres **saber si es posible** → or / any

En este problema quieres el **mínimo costo** → min.
En el problema de los dados que vimos antes querías **contar formas** → suma.
La operación la dicta el objetivo del problema, no la estructura.

Esta plantilla resuelve el 80% de los problemas de DP. Lo único que cambia es:
1.Qué es el "estado"
2.Cuáles son las "opciones"
3.Qué operación combina (min, max, +)

4.Cuál es el caso base
5. Aplícalo tú mismo a este problema
Estado: d = día actual.
Opciones desde d:
•Opción 1: pase de 1 día → cuesta costs[0], me lleva a d+1
•Opción 2: pase de 7 días → cuesta costs[1], me lleva a d+7
•Opción 3: pase de 30 días → cuesta costs[2], me lleva a d+30
Objetivo: minimizar costo.
Entonces:

```
f(d) = min(
    costs[0] + f(d + 1),
    costs[1] + f(d + 7),
    costs[2] + f(d + 30)
)
```

No hay magia. Es literalmente leer el enunciado y traducirlo.

6. ¿Y de dónde salen d+1, d+7, d+30?

Del **enunciado**:

- "Un pase de 1 día te permite viajar durante 1 día"
- "Un pase de 7 días te permite viajar durante 7 días"
- "Un pase de 30 días te permite viajar durante 30 días"

Si compras un pase de 7 días el día d, cubres los días d, d+1, ..., d+6. Por lo tanto, el siguiente día **sin cubrir** es d+7.
Por eso f(d + 7), no f(d + 6).

7. El truco para no equivocarte nunca
Antes de escribir código, escribe en papel:

```
f(d) = ?
        ↑
   ¿qué decisiones tengo aquí?
        ↓
   Opción A: costo X, me lleva a estado X'
   Opción B: costo Y, me lleva a estado Y'
   Opción C: costo Z, me lleva a estado Z'
        ↓
   ¿Qué quiero? ¿min, max, suma?
        ↓
   f(d) = min/max/suma( X + f(X'), Y + f(Y'), Z + f(Z') )
```

Si haces esto **antes** de programar, la solución sale sola. El error típico es intentar escribir el código primero y razonar después.

8. Ejemplos para que veas el patrón

Problema	Opciones	Combinación

Escalera (subir 1 o 2)	subir 1, subir 2	min (costo) o suma (formas)
Tickets de viaje	pase 1d, 7d, 30d	min
Dados (sumar objetivo)	cara 1..K	suma (contar)
Cambio de monedas	moneda 1, 5, 10	min (mínimas monedas)
Robar casas (no adyacentes)	robar, no robar	max

Todos tienen la misma estructura: estado + opciones + operación.

Resumen final

Para "percatarte" en próximos ejercicios:

1. **Identifica el estado** (¿qué información necesito para decidir?).
2. **Lista TODAS las decisiones posibles desde ese estado** (normalmente están en el enunciado).
3. **Para cada decisión, anota: cuánto cuesta y a qué nuevo estado me lleva.**
4. **Identifica el objetivo** (min, max, contar, etc.) → eso te da la operación.
5. **Escribe la fórmula**: f(estado) = operación(costo_i + f(nuevo_estado_i)).
6. **Define el caso base** (¿cuándo ya no puedo/debo decidir más?).

Ejercicio: Repartidor de pizzas

Eres un repartidor de pizzas. Tienes una lista `pedidos` con los **minutos del día** (0 a 1440) en los que debes entregar una pizza, ordenada de menor a mayor.
Cada entrega tarda exactamente **1 minuto** desde que llegas al cliente.
Puedes elegir entre **tres modos de transporte** para moverte entre entregas:

Modo	Costo (en $)	Velocidad
Bicicleta	`costos[0]`	1 minuto por unidad de distancia
Moto	`costos[1]`	Puedes cubrir **hasta 5 minutos** de viaje con un solo pago
Auto	`costos[2]`	Puedes cubrir **hasta 20 minutos** de viaje con un solo pago

Los modos "moto" y "auto" funcionan así: cuando los usas, **pagas una vez** y quedas cubierto para cualquier entrega cuya diferencia de tiempo desde la entrega actual sea **menor o igual** al límite del modo.
Es decir:
•Si usas moto en la entrega del minuto `t`, cubres todas las entregas en `[t, t+5]`.
•Si usas auto en la entrega del minuto `t`, cubres todas las entregas en `[t, t+20]`.
Debes entregar **todas** las pizzas, en orden. No puedes saltarte ninguna.
Devuelve el **costo mínimo** para completar todas las entregas.

Ejemplo 1
text
Copiar
Descargar
pedidos = [10, 20, 30]
costos = [3, 8, 15]
Salida: `9`
Explicación:
•Bicicleta en minuto 10 → cuesta 3
•Bicicleta en minuto 20 → cuesta 3
•Bicicleta en minuto 30 → cuesta 3
•Total: 9
(¿Podrías usar moto o auto y gastar menos? No, porque las entregas están muy separadas.)

Ejemplo 2
text
Copiar
Descargar
pedidos = [1, 2, 3, 4, 5, 6, 20]
costos = [3, 8, 15]
Salida: `11`
Explicación:
•Moto en minuto 1 → cubre entregas 1-6 → cuesta 8
•Bicicleta en minuto 20 → cuesta 3
•Total: 11
(Alternativa: ¿qué pasaría con auto en minuto 1? Cubre 1-21 → cuesta 15. Peor que 11.)

Ejemplo 3

text
Copiar
Descargar
pedidos = [1, 3, 5, 7, 9, 25]
costos = [2, 6, 12]
Salida: 10
Explicación:
•Bicicleta en minuto 1 → cuesta 2
•Moto en minuto 3 → cubre 3-8 → cuesta 6
•Bicicleta en minuto 9 → cuesta 2
•Bicicleta en minuto 25 → cuesta 2
•Total: 12...
Espera, eso da 12, pero la salida es 10. 🤔
(Pista: revisa si puedes agrupar mejor los pedidos con los modos disponibles.)

🧭 Tu misión

Antes de escribir código, responde en papel:
1.**¿Cuál es el estado?** (¿qué información necesitas para decidir?)
2.**¿Qué decisiones puedes tomar desde ese estado?** (las opciones)
3.**Para cada decisión: ¿cuánto cuesta y a qué nuevo estado te lleva?**
•Ojo: "a qué nuevo estado te lleva" depende de **cuántos pedidos cubre** el modo elegido.
4.**¿Qué quieres?** (min, max, contar...)
5.**Escribe la recurrencia.**
6.**¿Cuál es el caso base?**

🔍 Pista clave

Fíjate que el "nuevo estado" **no es simplemente** `t + algo`. Es:
"El índice del **primer pedido que NO queda cubierto** por el modo que elegí."
Eso es distinto al problema de los tickets, donde el nuevo estado era un día fijo (`d+1`, `d+7`, `d+30`). Aquí depende de **dónde caen los pedidos**.
Esa es la parte interesante: **el patrón es el mismo, pero el "nuevo estado" se calcula distinto**.

Ejercicio: Construir un muro

Debes construir un muro de **exactamente** `N` **metros de largo**. Tienes tres tipos de bloques para usar:

Bloque	Largo	Costo
Pequeño	`largos[0]` metros	`costos[0]`
Mediano	`largos[1]` metros	`costos[1]`
Grande	`largos[2]` metros	`costos[2]`

Puedes usar **cualquier cantidad** de cada tipo de bloque, en **cualquier orden**.
Debes cubrir **exactamente** los `N` metros (no puede sobrar ni faltar).
Devuelve el **costo mínimo** para construir el muro. Si es imposible cubrir exactamente `N` metros con los bloques disponibles, devuelve `-1`.

Ejemplo 1
N = 10
largos = [2, 3, 5]
costos = [3, 5, 8]
Salida: `16`
Explicación:
•Dos bloques grandes (5+5) → 8+8 = 16
•Alternativa: cinco bloques pequeños (2×5) → 3×5 = 15...
Hmm, eso da 15, que es mejor que 16. 🤔
(Pista: ¿de verdad puedes cubrir 10 con bloques de largo 2? Sí: 2+2+2+2+2 = 10. Entonces la respuesta correcta sería 15, no 16. Revisa bien.)

Ejemplo 2
N = 7
largos = [2, 4, 6]
costos = [3, 5, 7]
Salida: `-1`
Explicación:
•Con bloques de 2, 4 y 6, **todas las sumas posibles son pares**.
•7 es impar → imposible.
•Por eso `-1`.

Ejemplo 3
N = 11
largos = [1, 3, 5]
costos = [2, 4, 7]
Salida: `15`
Explicación:
•Un bloque de 5 + dos bloques de 3 → 7 + 4 + 4 = 15
•Alternativa: once bloques de 1 → 2×11 = 22 (peor)
•Alternativa: 5+5+1 → 7+7+2 = 16 (peor)
•La mejor es 15.

🧭 Tu misión

Antes de escribir código, responde en papel:

1. **¿Cuál es el estado?**
   - Pista: no es "qué bloque uso", es "cuánto me falta por cubrir".
2. **¿Qué decisiones puedes tomar desde ese estado?**
3. **Para cada decisión: ¿cuánto cuesta y a qué nuevo estado te lleva?**
4. **¿Qué quieres?** (min, max, contar...)
5. **Escribe la recurrencia.**
6. **¿Cuál es el caso base?**
   - Pista: hay **dos** casos base importantes aquí (uno cuando llegas a 0, otro cuando te pasas).
7. **¿Cómo manejas el `-1`?** (cuando es imposible)

🔍 Pista clave

Este problema se parece mucho al de los dados (contar formas), pero con dos diferencias:

- En vez de **contar**, quieres **minimizar**.
- En vez de **sumar exacto**, puede haber **estados imposibles** que debes representar con un valor especial (como infinito o `-1`).

Piensa: si `f(restante)` representa el costo mínimo de cubrir `restante` metros, ¿qué pasa cuando `restante < 0`? ¿Y cuando `restante == 0`?

🎯 Comparación con los anteriores

Problema	Estado	Opciones	Operación
Dados	(n dados, suma)	cara 1..K	suma
Tickets	día `d`	pase 1, 7, 30	min
Pizzas	índice pedido	bicicleta, moto, auto	min
Muro	**metros restantes**	**bloque 1, 2, 3**	**min**

Fíjate que la **estructura es idéntica**. Solo cambia el significado de cada pieza.

Ejercicio: Viaje por carretera

Estás haciendo un viaje por carretera de `D` **kilómetros** en línea recta. A lo largo del camino hay **gasolineras** en posiciones específicas (en kilómetros desde el inicio), dadas en una lista `estaciones` ordenada de menor a mayor.

Tu auto tiene un tanque con autonomía de `A` **kilómetros**. Empiezas con el tanque **lleno** en el kilómetro 0.

En cada gasolinera puedes decidir **cuántos litros cargar**, pero por simplicidad, cada gasolinera ofrece **tres opciones de carga**:

Opción	Litros	Costo
Carga pequeña	`litros[0]`	`costos[0]`
Carga media	`litros[1]`	`costos[1]`
Carga grande	`litros[2]`	`costos[2]`

Cada litro te da **1 km** de autonomía. El tanque tiene capacidad máxima de `A` **litros** (no puedes pasarte).

Debes llegar al kilómetro `D`. Si en algún momento te quedas sin gasolina antes de llegar, el viaje es imposible.

Devuelve el **costo mínimo** para llegar a destino. Si es imposible, devuelve `-1`.

Ejemplo 1
D = 100
A = 50
estaciones = [20, 50, 80]
litros = [10, 25, 40]
costos = [5, 12, 18]
Salida: 23
Explicación:
•Empiezas con 50 litros (autonomía hasta km 50).
•En km 20: cargas 25 litros → cuesta 12. Tanque: 30+25 = 55 → topado a 50.
•En km 50: cargas 25 litros → cuesta 12. Tanque: 0+25 = 25...
Hmm, eso no cuadra. Revisemos:
•Empiezas con 50 L. Llegas a km 20 con 30 L.
•Cargas 25 L → 55 L, topado a 50 L. Costo: 12.
•Llegas a km 50 con 20 L.
•Cargas 10 L → 30 L. Costo: 5. Total: 17.
•Llegas a km 80 con 0 L.
•Cargas 10 L → 10 L. Costo: 5. Total: 22.
•Llegas a km 100 con -10 L... ¡imposible!
(Pista: hay que planear mejor. Quizás cargar más en alguna estación. La respuesta correcta es 23, ¿puedes encontrar la combinación?)

Ejemplo 2
D = 200
A = 100
estaciones = [50, 120]
litros = [20, 50, 90]

costos = [8, 18, 30]
Salida: -1
Explicación:
•Empiezas con 100 L (autonomía hasta km 100).
•La siguiente estación está en km 50 → llegas con 50 L.
•Pero de km 50 a km 120 hay 70 km, y solo tienes 50 L + lo que cargues.
•Si cargas 90 L → 140 L, topado a 100 L → llegas a km 120 con 30 L.
•De km 120 a km 200 hay 80 km, pero solo tienes 30 L + lo que cargues.
•Si cargas 90 L → 120 L, topado a 100 L → llegas a km 200 con 20 L. ¡Sí se puede!
Espera, entonces **sí** se puede. ¿Por qué la salida es -1? 🤔
(Pista: revisa bien las distancias. ¿Hay algún tramo entre estaciones que supere la autonomía máxima?)

Ejemplo 3
D = 150
A = 60
estaciones = [30, 70, 110]
litros = [15, 30, 50]
costos = [6, 11, 17]
Salida: 28
Explicación:
•Empiezas con 60 L (autonomía hasta km 60).
•Km 30: llegas con 30 L. Cargas 30 L → 60 L. Costo: 11.
•Km 70: llegas con 20 L. Cargas 30 L → 50 L. Costo: 11.
•Km 110: llegas con 10 L. Cargas 15 L → 25 L. Costo: 6. Total: 28.
•Llegas a km 150 con 25 - 40 = -15 L... ¡imposible!
(Pista: hay que cargar más en algún punto. La respuesta es 28, ¿puedes encontrar la combinación correcta?)

🧭 Tu misión
Antes de escribir código, responde en papel:
1.**¿Cuál es el estado?**
•Pista: no basta con "en qué estación estoy". También importa **cuánta gasolina me queda**.
•Este es un estado **bidimensional**: (índice_estación, gasolina_restante).
2.**¿Qué decisiones puedes tomar desde ese estado?**
•Tres opciones de carga (o quizás también "no cargar").
3.**Para cada decisión: ¿cuánto cuesta y a qué nuevo estado te lleva?**
•El nuevo estado depende de **cuánta gasolina gastas** en llegar a la siguiente estación.
4.**¿Qué quieres?** (min, max, contar...)
5.**Escribe la recurrencia.**
6.**¿Cuáles son los casos base?**
•Llegar a destino → costo 0.
•Quedarte sin gasolina antes de llegar → imposible (∞).
•No poder llegar a la siguiente estación → imposible.

🔍 Pista clave
Este problema introduce **dos novedades** respecto a los anteriores:

1.**Estado bidimensional:** no basta con "dónde estoy", también importa "con cuánto llego".
2.**Restricción de capacidad:** no puedes cargar más de `A` litros.
Pero la **estructura es la misma**: estado + opciones + costo + nuevo estado.

🎯 Comparación con los anteriores

Problema	Estado	Opciones	Operación
Dados	(n dados, suma)	cara 1..K	suma
Tickets	día `d`	pase 1, 7, 30	min
Pizzas	índice pedido	bicicleta, moto, auto	min
Muro	metros restantes	bloque 1, 2, 3	min
Vacaciones	día `d`	paquete 1d, 7d, 30d	min
Gasolina	**(estación, gasolina)**	**carga S, M, L**	**min**

¿Ves cómo el patrón se repite una y otra vez? Eso es lo que debes internalizar.

Ejercicio: Maratón de series

Quieres ver una serie completa que tiene N **episodios** en total, numerados del 1 al N. Cada episodio dura duraciones[i] minutos.
Tienes D **días libres** para verla. Cada día puedes ver **como máximo** M **minutos** de serie (porque tienes otras cosas que hacer).
Puedes ver los episodios **en orden**, sin saltarte ninguno. Un episodio **no se puede partir**: si empiezas a verlo, lo terminas el mismo día (aunque te pases un poco del límite, no se puede).
Cada día puedes decidir:
•**Ver episodios** hasta que ya no quepa el siguiente.
•**Descansar** (no ver nada ese día).
Devuelve **cuántos episodios como máximo** puedes ver en total. Si puedes verlos todos, devuelve N.

Ejemplo 1
N = 5
D = 3
M = 100
duraciones = [30, 40, 50, 60, 70]
Salida: 5
Explicación:
•Día 1: episodios 1, 2 → 30 + 40 = 70 ≤ 100
•Día 2: episodios 3, 4 → 50 + 60 = 110 > 100. Solo episodio 3 → 50.
•Día 3: episodio 4 → 60.
Espera, eso da solo 4 episodios (1, 2, 3, 4). ¿Puedes ver el 5? 🤔
(Pista: quizás conviene no ver el episodio 4 el día 2, y reorganizar. La respuesta correcta es 5, ¿cómo?)

Ejemplo 2
N = 4
D = 2
M = 50
duraciones = [30, 30, 30, 30]
Salida: 2
Explicación:
•Día 1: episodio 1 → 30 ≤ 50. No cabe el 2 (60 > 50).
•Día 2: episodio 2 → 30 ≤ 50. No cabe el 3 (60 > 50).
•Total: 2 episodios.
No importa cómo lo organices, solo puedes ver 1 episodio por día porque 2 juntos superan M. Con 2 días, máximo 2 episodios.
Ejemplo 3
N = 6
D = 3
M = 10
duraciones = [4, 5, 6, 3, 2, 5]
Salida: 6
Explicación:
•Día 1: episodios 1, 2 → 4 + 5 = 9 ≤ 10. No cabe el 3 (15 > 10).
•Día 2: episodios 3, 4 → 6 + 3 = 9 ≤ 10. No cabe el 5 (11 > 10).

•Día 3: episodios 5, 6 → 2 + 5 = 7 ≤ 10.
•Total: 6 episodios. ¡Se pueden ver todos!

🧭 Tu misión

Antes de escribir código, responde en papel:

1.**¿Cuál es el estado?**
•Pista: necesitas saber **qué episodio sigue** y **cuántos días te quedan**.
•Estado bidimensional: `(episodio_actual, dias_restantes)`.

2.**¿Qué decisiones puedes tomar desde ese estado?**
•Pista: cada día puedes elegir **cuántos episodios ver** (0, 1, 2, ... hasta que ya no quepan).

3.**Para cada decisión: ¿cuánto "cuesta" y a qué nuevo estado te lleva?**
•Aquí el "costo" no es dinero, es **episodios vistos**. Quieres **maximizar**.
•El nuevo estado es `(episodio_actual + k, dias_restantes - 1)` donde `k` es cuántos viste ese día.

4.**¿Qué quieres?** (min, max, contar...)
•**Maximizar** episodios vistos.

5.**Escribe la recurrencia.**

6.**¿Cuáles son los casos base?**
•Si `dias_restantes == 0` → no puedes ver más.
•Si `episodio_actual > N` → ya viste todos.
•Si no cabe ningún episodio ese día → ese día no ves nada (equivale a descansar).

🔍 Pista clave

Aquí hay **dos novedades** respecto a los anteriores:

1.**La operación es** `max`**, no** `min` — porque quieres **maximizar** episodios, no minimizar costo.

2.**Las opciones no son fijas** — no son "3 paquetes" o "3 bloques". Son **"¿cuántos episodios veo hoy?"**, que puede ser 0, 1, 2, 3, ... hasta donde quepan.

Pero la **estructura es idéntica**: estado + opciones + transición + operación.

🎯 Comparación con los anteriores

Problema	Estado	Opciones	Operación
Dados	(n dados, suma)	cara 1..K	suma
Tickets	día `d`	pase 1, 7, 30	min
Pizzas	índice pedido	bicicleta, moto, auto	min
Muro	metros restantes	bloque 1, 2, 3	min
Vacaciones	día `d`	paquete 1d, 7d, 30d	min
Gasolina	(estación, gasolina)	carga S, M, L	min
Series	**(episodio, días)**	**ver k episodios hoy**	**max**

Fíjate en la última fila: **misma estructura, pero** `max` **en vez de** `min`**, y opciones variables en vez de fijas.**

Eso es lo que debes entrenar: **no memorizar problemas, sino reconocer el patrón**.

Ejercicio: Speedrun de videojuego

Estás haciendo un **speedrun** de un videojuego que tiene N **niveles** en total, numerados del 1 al N. Cada nivel tiene una **dificultad** dificultades[i].

Tienes una **barra de energía** que empieza al **máximo** E y se **agota** con cada nivel que juegas. La energía que gasta el nivel i es exactamente dificultades[i].

Cuando te quedas sin energía suficiente para el siguiente nivel, tienes **dos opciones**:

•**Tomar una poción** (solo tienes P pociones en total). Cada poción **restaura la energía al máximo** E.

•**Terminar el speedrun** (dejas de jugar y el run termina ahí).

Puedes tomar poción **en cualquier momento**, incluso antes de empezar un nivel, pero **no puedes tener más energía que** E.

Cada nivel que completas te da **1 punto**. Quieres **maximizar** el número de niveles completados.

Devuelve el **máximo de niveles** que puedes completar.

Ejemplo 1

N = 5

E = 10

P = 1

dificultades = [4, 5, 6, 3, 2]

Salida: 5

Explicación:

•Nivel 1 (4): energía 10 → 6

•Nivel 2 (5): energía 6 → 1

•Nivel 3 (6): no alcanza (1 < 6). Tomas poción → energía 10 → 4

•Nivel 4 (3): energía 4 → 1

•Nivel 5 (2): no alcanza (1 < 2). No quedan pociones → termina.

•Total: 4 niveles.

Espera, eso da 4, pero la salida es 5. 🤔

(Pista: ¿conviene tomar la poción antes, en otro momento? La respuesta es 5, ¿cómo?)

Ejemplo 2

N = 4

E = 8

P = 0

dificultades = [3, 3, 3, 3]

Salida: 2

Explicación:

•Nivel 1 (3): energía 8 → 5

•Nivel 2 (3): energía 5 → 2

•Nivel 3 (3): no alcanza (2 < 3). No hay pociones → termina.

•Total: 2 niveles.

Ejemplo 3

N = 6

E = 12

P = 2

dificultades = [5, 5, 5, 5, 5, 5]

Salida: 6

Explicación:

•Nivel 1 (5): energía 12 → 7

•Nivel 2 (5): energía 7 → 2

•Nivel 3 (5): no alcanza. Poción → 12 → 7
•Nivel 4 (5): energía 7 → 2
•Nivel 5 (5): no alcanza. Poción → 12 → 7
•Nivel 6 (5): energía 7 → 2
•Total: 6 niveles. ¡Se pueden completar todos!

🧭 Tu misión

Antes de escribir código, responde en papel:
1.**¿Cuál es el estado?**
•Pista: necesitas saber **qué nivel sigue**, **cuánta energía queda** y **cuántas pociones te quedan**.
•Estado **tridimensional**: (nivel_actual, energia_restante, pociones_restantes).
2.**¿Qué decisiones puedes tomar desde ese estado?**
•Pista: en cada nivel puedes:
•Jugar el nivel (si tienes energía suficiente).
•Tomar poción (si te quedan) y luego jugar.
•Terminar el run.
3.**Para cada decisión: ¿cuánto "cuesta" y a qué nuevo estado te lleva?**
•El "costo" no es dinero, es **niveles completados**. Quieres **maximizar**.
4.**¿Qué quieres?** (min, max, contar...)
•**Maximizar** niveles completados.
5.**Escribe la recurrencia.**
6.**¿Cuáles son los casos base?**
•Si nivel_actual > N → ya completaste todos.
•Si no puedes jugar ni tomar poción → terminas el run.
•Si decides terminar → devuelves 0 desde ahí.

🔍 Pista clave

Este problema tiene **dos novedades** importantes:
1.**Estado tridimensional** — antes tenías 1 o 2 dimensiones; ahora son 3: nivel, energía y pociones.
2.**Decisiones anidadas** — "tomar poción" y "jugar nivel" pueden ir juntas o separadas. Debes modelar bien si tomas poción **antes** o **después** de decidir jugar.
Pero la **estructura es la misma**: estado + opciones + transición + operación.

🎯 Comparación con los anteriores

Problema	Estado	Opciones	Operación
Dados	(n dados, suma)	cara 1..K	suma
Tickets	día `d`	pase 1, 7, 30	min
Pizzas	índice pedido	bicicleta, moto, auto	min
Muro	metros restantes	bloque 1, 2, 3	min
Vacaciones	día `d`	paquete 1d, 7d, 30d	min
Gasolina	(estación, gasolina)	carga S, M, L	min
Series	(episodio, días)	ver k episodios hoy	max
Speedrun	**(nivel, energía, pociones)**	**jugar, poción, terminar**	**max**

Fíjate cómo va creciendo la complejidad del **estado** (1D → 2D → 3D), pero la **estructura no cambia**.

🧠 Lección importante

Cuando el estado crece en dimensiones, el problema **no se vuelve conceptualmente distinto**. Sigue siendo:
"¿Dónde estoy? ¿Qué puedo hacer? ¿A dónde voy? ¿Qué quiero?"
La diferencia es solo **cuánta información necesito recordar** para no repetir cálculos.
Eso es exactamente lo que hace la programación dinámica: **recordar** en lugar de **recalcular**.

746- **Min Cost Climbing Stairs**

https://leetcode.com/problems/min-cost-climbing-stairs/description/

Se te da un arreglo de enteros `cost` donde `cost[i]` es el costo del i-ésimo escalón de una escalera.
Una vez que pagas el costo, puedes subir uno o dos escalones.
Puedes comenzar desde el escalón con índice 0, o desde el escalón con índice 1.
Devuelve el costo mínimo para llegar a la cima de la escalera, que es la posición justo después del último escalón (índice `cost.length`).
Ejemplo 1:
Entrada: cost = [10,15,20]
Salida: 15
Explicación: Comenzarás en el índice 1.
•Pagas 15 y subes dos escalones para llegar a la cima.
El costo total es 15.
Ejemplo 2:
Entrada: cost = [1,100,1,1,1,100,1,1,100,1]
Salida: 6
Explicación: Comenzarás en el índice 0.
•Pagas 1 y subes dos escalones para llegar al índice 2.
•Pagas 1 y subes dos escalones para llegar al índice 4.
•Pagas 1 y subes dos escalones para llegar al índice 6.
•Pagas 1 y subes un escalón para llegar al índice 7.
•Pagas 1 y subes dos escalones para llegar al índice 9.
•Pagas 1 y subes un escalón para llegar a la cima.
El costo total es 6.
Restricciones:
2 <= cost.length <= 1000
0 <= cost[i] <= 999

3693- Climbing Stairs II https://leetcode.com/problems/climbing-stairs-ii/

Estás subiendo una escalera con **n + 1** escalones, numerados del 0 al n.
También se te da un arreglo de enteros costs indexado en 1 de longitud n, donde **costs[i]** es el costo del escalón i.
Desde el escalón i, solo puedes saltar al escalón **i + 1**, **i + 2**, o **i + 3**. El costo de saltar del escalón i al escalón j se define como: **costs[j] + (j - i)^2**
Comienzas desde el escalón 0 con costo = 0.
Devuelve el costo total mínimo para llegar al escalón n.
Ejemplo 1:
Entrada: n = 4, **costs** = [1,2,3,4]
Salida: 13
Explicación:
Un camino óptimo es 0 → 1 → 2 → 4

Salto	Cálculo del Costo	Costo
0 → 1	costs[1] + (1 - 0)^2 = 1 + 1	2
1 → 2	costs[2] + (2 - 1)^2 = 2 + 1	3
2 → 4	costs[4] + (4 - 2)^2 = 4 + 4	8

Por lo tanto, el costo total mínimo es 2 + 3 + 8 = 13
Ejemplo 2:
Entrada: n = 4, costs = [5,1,6,2]
Salida: 11
Explicación:
Un camino óptimo es 0 → 2 → 4

Salto	Cálculo del Costo	Costo
0 → 2	costs[2] + (2 - 0)^2 = 1 + 4	5
2 → 4	costs[4] + (4 - 2)^2 = 2 + 4	6

Por lo tanto, el costo total mínimo es 5 + 6 = 11
Ejemplo 3:
Entrada: n = 3, costs = [9,8,3]
Salida: 12
Explicación:
El camino óptimo es 0 → 3 con costo total = costs[3] + (3 - 0)^2 = 3 + 9 = 12
Restricciones:
1 <= n == costs.length <= 10^5
1 <= costs[i] <= 10^4

983-Minimum Cost for Tickets **https://leetcode.com/problems/minimum-cost-for-tickets/**

https://neetcode.io/solutions/minimum-cost-for-tickets

Has planeado algunos viajes en tren con un año de anticipación. Los días del año en los que viajarás se dan como un arreglo de enteros days. Cada día es un entero del 1 al 365.

Los boletos de tren se venden de tres formas diferentes:

- un pase de 1 día se vende por costs[0] dólares,
- un pase de 7 días se vende por costs[1] dólares, y
- un pase de 30 días se vende por costs[2] dólares.

Los pases permiten esa cantidad de días de viaje consecutivos.

- Por ejemplo, si obtenemos un pase de 7 días el día 2, entonces podemos viajar durante 7 días: 2, 3, 4, 5, 6, 7 y 8.

Devuelve la cantidad mínima de dólares que necesitas para viajar cada día en la lista de días dada.

Ejemplo 1:

Entrada: days = [1,4,6,7,8,20], costs = [2,7,15]

Salida: 11

Explicación: Por ejemplo, aquí hay una forma de comprar pases que te permite viajar según tu plan de viaje:

El día 1, compraste un pase de 1 día por costs[0] = $2, que cubrió el día 1.

El día 3, compraste un pase de 7 días por costs[1] = $7, que cubrió los días 3, 4, ..., 9.

El día 20, compraste un pase de 1 día por costs[0] = $2, que cubrió el día 20.

En total, gastaste $11 y cubriste todos los días de tu viaje.

Ejemplo 2:

Entrada: days = [1,2,3,4,5,6,7,8,9,10,30,31], costs = [2,7,15]

Salida: 17

Explicación: Por ejemplo, aquí hay una forma de comprar pases que te permite viajar según tu plan de viaje:

El día 1, compraste un pase de 30 días por costs[2] = $15 que cubrió los días 1, 2, ..., 30.

El día 31, compraste un pase de 1 día por costs[0] = $2 que cubrió el día 31.

En total, gastaste $17 y cubriste todos los días de tu viaje.

Restricciones:

1 <= days.length <= 365

1 <= days[i] <= 365

days está en orden estrictamente creciente.

costs.length == 3

1 <= costs[i] <= 1000

Vacation https://vjudge.net/problem/AtCoder-dp_c

Las vacaciones de verano de Taro comienzan mañana, y ha decidido hacer planes ahora.

Las vacaciones consisten en **N** días. Para cada **i** ($1 \leq i \leq N$), Taro elegirá una de las siguientes actividades y la hará en el **i**-ésimo día:

- **A**: Nadar en el mar. Gana $\mathbf{a_i}$ puntos de felicidad.
- **B**: Atrapar insectos en las montañas. Gana $\mathbf{b_i}$ puntos de felicidad.
- **C**: Hacer la tarea en casa. Gana $\mathbf{c_i}$ puntos de felicidad.

Como Taro se aburre fácilmente, **no puede hacer las mismas actividades durante dos o más días consecutivos**.

Encuentra el **máximo total posible de puntos de felicidad** que Taro puede ganar.

Restricciones

- Todos los valores en la entrada son números enteros.
- $1 \leq N \leq 10^5$
- $1 \leq a_i, b_i, c_i \leq 10^4$

Entrada

La entrada se da desde la entrada estándar con el siguiente formato:

```
N
a1 b1 c1
a2 b2 c2
:
:
a_N b_N c_N
```

Salida

Imprime el **máximo total posible de puntos de felicidad** que Taro gana.

Ejemplo 1

Entrada:

```
3
10 40 70
20 50 80
30 60 90
```

Salida:

```
210
```

Si Taro hace las actividades en el orden **C, B, C**, ganará **70** + **50** + **90** = **210** puntos de felicidad.

Ejemplo 2

Entrada:

```
1
100 10 1
```

Salida:

```
100
```

Ejemplo 3

Entrada:

```
7
6 7 8
```

8 8 3
2 **5** 2
7 8 6
4 6 **8**
2 **3** 4
7 5 1

Salida:

46

Taro debería hacer las actividades en el orden **C, A, B, A, C, B, A**.